P10-8
Abstract Engine

Aurelius Ng (2500612)
Lim Jun Ming (2503352)
Long Chay Han (2500596)
Ian Sim Wei Xuan (2500159)
Ngian Hui Yang (2504000)
Zakir Hussain Rumaana Famin (2503352)

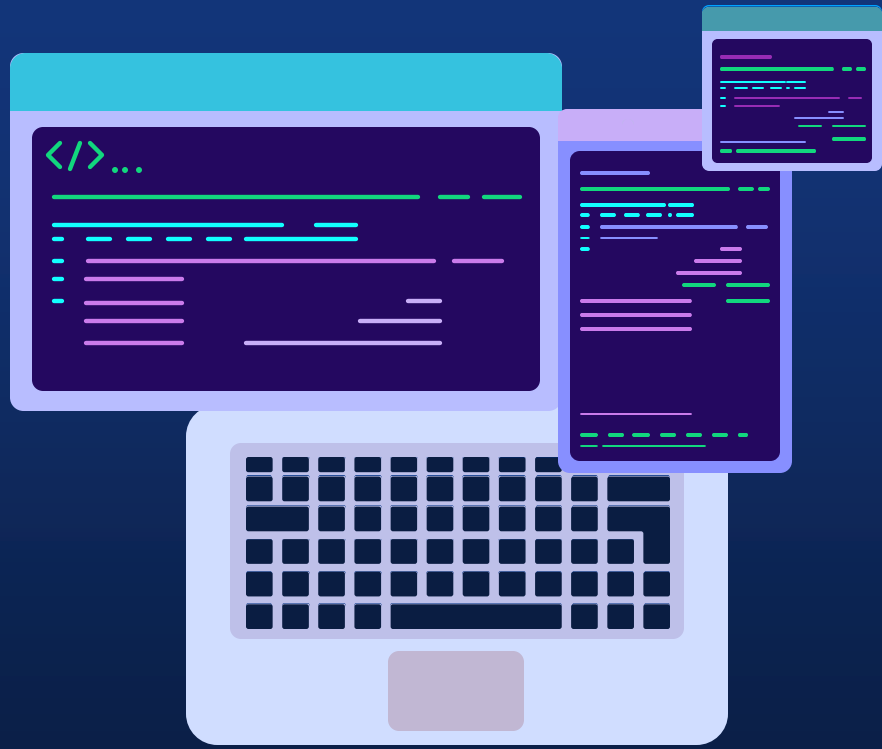

TABLE OF CONTENTS

01
KEY ASPECTS

02
UML DESIGN

03
LIVE DEMO

04
CODE STRUCTURE OVERVIEW

05
IMPLEMENTATION DETAILS

4.2 INDIVIDUAL CODE STRUCTURE OVERVIEW - RUMAANA

Entity System

Entity (Abstract Base Class)

Purpose:
Base class for all game objects which defines identity, transformation, and state.

Core Attributes:
Id, position rotation, scale, active, Visible, kinematic, mass, velocity

Key Methods:
update() → Enforces behaviour
renderShapes() / renderTextures() → Rendering separation
setKinematic() → Physics control flexibility

```
public Entity(String id, float x, float y) {  langunforlife +1
    this.id = id;
    this.position = new Vector2(x, y);
    this.scale = new Vector2( x: 1,  y: 1);
    this.rotation = 0f;
    this.active = true;
    this.visible = true;
    this.velocity = new Vector2( x: 0,  y: 0);
}

// Abstract methods: Children MUST implement these
public abstract void update(float deltaTime);  5 implementations

// Generic render, specific renderers used in subclasses
public abstract void renderShapes(ShapeRenderer renderer);  6

public abstract void renderTextures(SpriteBatch batch);  5 impl
```

CollidableEntity (Abstract)

Purpose:
Handles physics and collision interactions.
Data Managed:
hitbox – Collision boundary
collisionLayer – Interaction group

Key Logic:
checkCollision()
onCollisionEnter()

```
public abstract class CollidableEntity extends Entity {  36 usages  langunforlife
    private Rectangle hitbox; //  2 usages
    private int collisionLayer; //  no usages

    public CollidableEntity(String id, float x, float y, float w, float h) {
        super(id, x, y);
        this.hitbox = new Rectangle(x, y, w, h);
    }
    // Give this method a BODY { } so children don't have to implement it
    public Rectangle getHitbox() {  14 usages  rumxxnx
        return hitbox;
    }
    public abstract boolean checkCollision(CollidableEntity other); //  no usa
    public abstract void onCollisionEnter(CollidableEntity other); //  2 usage
```

4.2 INDIVIDUAL CODE STRUCTURE OVERVIEW - RUMAANA

Entity System

NonCollidableEntity (Abstract)

Purpose:
Visual-only objects

Used For:
Backgrounds, UI elements, Labels

Components:
TextureObject – Displays images
TextLabel – Displays text

```
 */
public abstract class NonCollidableEntity extends Entity {  2 usages  2 inheritors  lang

    public NonCollidableEntity(String id, float x, float y) {  2 usages  langunforlife
        super(id, x, y);
    }

}
```

Shape (Abstract) - Subclass of CollidableEntity

Purpose:
Collidable geometric primitives with visual properties.

Attributes:
color: Color
filled: Boolean

Concrete Implementations:
CircleEntity → radius
TriangleEntity → vertices
RectangleEntity → width, height

```
@Override  Decoraze +1
public void renderShapes(ShapeRenderer renderer) {
    renderer.setColor(color);
    // Draw relative to position
    float size = 25;
    renderer.triangle(
            position.x,  y1: position.y + size, // top
            position.x - size,  y2: position.y - size, // bottom-left
            position.x + size, position.y - size // bottom-right
    );
}
```

```
public abstract class Shape extends CollidableEntity {  3 usages  3 inheritors
    protected Color color; //  4 usages
    protected boolean filled; //  2 usages

    public Shape(String id, float x, float y, float w, float h, Color color)
        super(id, x, y, w, h);
        this.color = color;
        this.filled = true;
    }

    public abstract void renderShapes(ShapeRenderer renderer);  3 implementatior
}
```

4.2 INDIVIDUAL CODE STRUCTURE OVERVIEW - IAN

Entity Manager

Role: Manages all entities

- Stores all entitles in a list
- Implements iEntityOps

Movement Manager

Role: Manages movements

- Implements iMovement
- Consist of 2 types of movement
    - PlayerMovement
    - AIMovement
- Maps Entity ID with AIMovement

OOP Concept: Polymorphism, Abstraction

4.2 INDIVIDUAL CODE STRUCTURE OVERVIEW – JUN MING

- Collision Manager
  - Implements iCollision containing checkCollisions
  - checkCollisions() (key method)
  - Uses CollisionDetection and CollisionResponse to do checks and calculations
  - Uses reverseDirection() in iMovement to tell MovementManager how to bounce the entity

```java
public interface iMovement {

    /**
     * Apply movement to an entity based on its velocity
     *
     * @param entity    The entity to move
     * @param deltaTime Time since last frame
     */
    void applyMovement(Entity entity, float deltaTime);

    /**
     * Apply physics calculations (gravity, acceleration, etc.)
     *
     * @param entity    The entity to apply physics to
     * @param deltaTime Time since last frame
     */
    void applyPhysics(Entity entity, float deltaTime);

    void applyPlayerMovement(Entity entity, float dt, iInput input);

    // allows CollisionManager to tell MovementManager to bounce an entity's
    // direction on collision.
    void reverseDirection(String entityId, float nx, float ny);// billiard balls lookign interaction when colliding.

}
```

```java
public interface iCollision {

    /**
     * Check if two collidable entities are colliding
     *
     * @param e1 First entity
     * @param e2 Second entity
     * @return true if colliding, false otherwise
     */
    boolean checkCollision(CollidableEntity e1, CollidableEntity e2);

}
```

```java
public void checkCollisions(List<CollidableEntity> collidableEntities) {
    // Step 1: Update cooldowns
    List<String> expired = new ArrayList<>();
    for (java.util.Map.Entry<String, Float> entry : collisionCooldowns.entrySet()) {
        entry.setValue(entry.getValue() - 0.016f);
        if (entry.getValue() <= 0)
            expired.add(entry.getKey());
    }
    for (String key : expired)
        collisionCooldowns.remove(key);

    // Step 2: Detect colliding pairs (delegated to CollisionDetection)
    List<CollidableEntity[]> pairs = detector.detectCollisions(collidableEntities);

    // Step 3: For each pair — notify + resolve
    for (CollidableEntity[] pair : pairs) {
        CollidableEntity e1 = pair[0];
        CollidableEntity e2 = pair[1];

        String key = e1.getId() + "<->" + e2.getId();
        logCollisions(e1, e2, key);

        e1.onCollisionEnter(e2);
        e2.onCollisionEnter(e1);

        // Reflect ONCE — billiard bounce
        float dx = e1.getPosition().x - e2.getPosition().x;
        float dy = e1.getPosition().y - e2.getPosition().y;
        float dist = (float) Math.sqrt(dx * dx + dy * dy);
        if (dist > 0) {
            float nx = dx / dist;
            float ny = dy / dist;
            movement.reverseDirection(e1.getId(), nx, ny);
            movement.reverseDirection(e2.getId(), -nx, -ny);
        }
        // Push apart every frame they overlap
        responder.resolveCollision(e1, e2);
    }
}
```

4.2 INDIVIDUAL CODE STRUCTURE OVERVIEW – JUN MING

- CollisionDetection
  - Takes a list of CollidableEntities
  - Loops through a list of collidable entities
  - Checks if any 2 entity is overlapping
  - If so the pair is added into an array list that detectCollision returns

```java
public class CollisionDetection {
    // DELETED: private List<CollidableEntity> collidables = new ArrayList<>();
    // DELETED: public void addCollidable(CollidableEntity e) { collidables.add(e);
    // }
    // DELETED: public void removeCollidable(String id) { ... }
    // made it simpler by decoupling (check with your branch see diff lah la haha)
    public List<CollidableEntity[]> detectCollisions(List<CollidableEntity> collidables) {// get collidable lists from
                                                                                          // collision manager
                                                                                          // detection just does the
                                                                                          // math SRP

        List<CollidableEntity[]> pairs = new ArrayList<>();
        for (int i = 0; i < collidables.size(); i++) {
            for (int j = i; j < collidables.size(); j++) {
                if (collidables.get(i).getHitbox().overlaps(collidables.get(j).getHitbox())) {
                    pairs.add(new CollidableEntity[] { collidables.get(i), collidables.get(j) });
                }
            }
        }
        return pairs;// instead of using two .get and add methods, we combined them to have a array
                     // returning back to the function. this is cleaner as its a flat list rather
                     // than individually
    }
}
```

4.2 INDIVIDUAL CODE STRUCTURE OVERVIEW – JUN MING

- CollisionResponse.resolveCollision()
  - Takes a pair of CollidableEntities
  - Calculate how much force is applied to collided Entities with respect to individual mass of each Entity

```java
public class CollisionResponse {
    public void resolveCollision(CollidableEntity e1, CollidableEntity e2) {
        float dx = e1.getPosition().x - e2.getPosition().x;
        float dy = e1.getPosition().y - e2.getPosition().y;
        float dvx = e1.getVelocity().x - e2.getVelocity().x;
        float dvy = e1.getVelocity().y - e2.getVelocity().y;
        float dist = (float) Math.sqrt(dx * dx + dy * dy);
        float totalMass = e1.getMass() + e2.getMass();
        float pushStrength1 = e1.getMass() / totalMass;
        float pushStrength2 = e2.getMass() / totalMass;
        if (dist > 0) {
            float nx = dx / dist;
            float ny = dy / dist;
            float relativeSpeed = (dvx * nx) + (dvy * ny);
            // these here are the kinematic checks (stuff is in entity)
            // kinematics is basically if the entity is a kinematic entity like the player,
            // it doesnt get pushed
            // we use += and -= here for entity 1 and entity 2 to ensure they go opposite
            // directions
            if (!e1.isKinematic()) {
                e1.getPosition().x += relativeSpeed * pushStrength1;
                e1.getPosition().y += relativeSpeed * pushStrength1;


            }
            if (!e2.isKinematic()) {
                e2.getPosition().x -= relativeSpeed * pushStrength2;
                e2.getPosition().y -= relativeSpeed * pushStrength2;
            }
        }
    }
}
```

4.2 INDIVIDUAL CODE STRUCTURE OVERVIEW – JUN MING

- CheckCollision()
  - Overlap check of a pair of CollidableEntities to check if they have collided

```java
public boolean checkCollision(CollidableEntity e1, CollidableEntity e2) {
    return e1.getHitbox().overlaps(e2.getHitbox());
}
```

- LogCollisions()
  - Prints collision logs in console

```java
public void logCollisions(CollidableEntity e1, CollidableEntity e2, String key) {
    if (!collisionCooldowns.containsKey(key)) {
        System.out.println("[CollisionManager] Collision detected: " + e1.getId() + " <-> " + e2.getId());
        collisionCooldowns.put(key, COOLDOWN);
    }

}
```

4.2 INDIVIDUAL CODE STRUCTURE OVERVIEW - HUI YANG

Components:
- ***SceneManager*** (Controller)
- ***Scene*** (Abstract base for all scenes)
- ***ISceneControl*** (Interface contract for other modules)

Purpose
- Controls active scene (First, Second, Third, Help)
- Ensures only active scene runs its update() and rendering methods
- Handles scene lifecycle
  (unload active scene → switch scene → load next scene)

OOP Concept(s):
- **Abstraction:** Scene defines required behavior without implementation (onLoad, update, render methods)
- **Aggregation:** SceneManager receives Scene objects via registerScene(), doesn't create them. They exist independently.
- **Polymorphism:** SceneManager stores different scene types as Scene and calls common methods
- **Encapsulation:** switching logic + current-scene tracking hidden inside SceneManager
- **Interface usage:** iSceneControl provides a clean contract for other systems to request scene changes

```java
protected Scene(
        String name,
        iCollision collision,
        iEntityOps entityOps,
        iSceneControl sceneCtrl,
        iInput input,
        iMovement movement,
        iAudio audio) {
    this.name = name;
    this.collision = collision;
    this.entityOps = entityOps;
    this.sceneCtrl = sceneCtrl;
    this.input = input;
    this.audio = audio;
    this.movement = movement;
}
```

```java
// Unload current
if (currentScene != null) {
    currentScene.unload();
}

// Switch
currentScene = next;
currentSceneName = sceneName;

// Load next
currentScene.load();

System.out.println("[SceneManager] Switched to: " + currentSceneName);
```

```java
public interface iSceneControl {

    void switchScene(String sceneName);

    Scene getCurrentScene();

    String getCurrentSceneName();
}
```

4.2 INDIVIDUAL CODE STRUCTURE OVERVIEW – CHAY HAN

Input Output Manager

Role: Handles keyboard/mouse input and audio output

- 6 key methods to detect Input from Mouse and 3 key methods to play the output audio

iInput Interface

IsKeyPressed()
IsKeyJustPressed()
GetMousePosition()
IsMouseButtonPressed()

```
public interface iInput {

    /** Check if a specific key is currently pressed ...*/
    boolean isKeyPressed(int keyCode);  5 usages  1 implementation

    /** Check if a specific key was just pressed this frame ...*/
    boolean isKeyJustPressed(int keyCode);  10 usages  1 implementation

    /** Get the current mouse/touch position ...*/
    Vector2 getMousePosition();  1 usage  1 implementation

    /** Check if mouse button is pressed ...*/
    boolean isMouseButtonPressed(int button);  no usages  1 implementation

    /** Play a sound effect ...*/

    // void playSound(String soundName);

    /** Play background music ...*/
    // void playMusic(String musicName);

    /** Stop all currently playing music */
    // void stopMusic();
}
```

iAudio Interface

PlaySound()
PlayMusic()
StopMusic()

```
public interface iAudio {  16 usages  1 implementation
    void playSound(String soundName);  2 usages  1 implementation

    void playMusic(String musicName);  1 usage  1 implementation

    void stopMusic();  1 usage  1 implementation
}
```

THANK YOU